



FORMATION INFORMATIQUE - EQUIPE

Git - En equipe

Flux, revue, rebase, CI et releases :
collaborer sans se marcher dessus.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Rappel rapide et carte du livre

- Ce que tu gardes en tete
- La carte du livre
- Un fil rouge
- Ce dont tu as besoin
- Comment lire ce livre
- Erreur classique
- En vrai
- A toi

2. Chapitre 2 - Flux d'equipe : qui pousse ou, quand tirer

- L'image du tableau blanc
- Une journee type a trois
- Qui pousse ou ?
- Quand tirer (pull) ?
- Communication minimale
- Le rythme "petit et frequent"
- Que faire si tu es bloque par quelqu'un ?
- Le remote n'est pas magique
- Matin, midi, merge
- Fichiers chauds
- Erreur classique
- En vrai
- A toi

3. Chapitre 3 - Strategie de branches simple (main + feature)

- Le modele simple : main + features
- Noms de branches qui aident
- Combien de temps une branche vit-elle ?
- Et develop ? Et release ?
- Une branche par intention
- Branches partagees
- Hotfix : le petit frere du fix
- Nettoyage
- Carte mentale pour un stagiaire
- Erreur classique
- En vrai
- A toi

4. Chapitre 4 - Rebase vs merge : idee, quand, risques

- L'idee en une image
- Un cas concret : Lea met a jour sa feature
- Quand le rebase aide
- Quand le merge est plus simple (et plus sur)

- Le risque numero un : reecire l'histoire partagee
- Conflits : meme stress, meme methode
- Sans dogme : une politique d'equipe possible
- Erreur classique
- En vrai
- A toi

5. Chapitre 5 - Historique propre : messages, petits commits, squash

- Des messages qui racontent le pourquoi
- Petits commits, grandes PR ? Non : petits commits et PR raisonnables
- L'idee du squash
- Amend : corriger le tout dernier commit (avec prudence).
- Ce qu'on ne met pas dans l'historique
- Relire avant de demander la review
- Conventional commits ? Optionnel
- Erreur classique
- En vrai
- Un apres-midi chez Lea, Max et Sam
- Relire le diff comme un inconnu
- Fichiers qui ne devraient pas etre la
- Quand WIP est acceptable
- A toi

6. Chapitre 6 - Revue de code humaine

- Pourquoi reviewer ?
- Quoi regarder (sans tout relire ligne a ligne).
- Le ton qui aide
- En tant qu'auteur de la PR
- But
- Changements
- Comment tester
- Taille de PR
- Temps de reponse
- Approve, request changes, commentaire
- Erreur classique
- En vrai
- Exemple de fil de review (ton juste).
- Ce qu'on ne review pas (ou peu).
- Check-list mentale du reviewer (sans en faire une religion).
- Quand tu n'es pas expert du domaine
- A toi

7. Chapitre 7 - Branches protegees : pas de push direct sur main

- L'idee
- Ou ca se regle (idee GitHub).
- Ce que ca change au quotidien
- Administrateurs et exceptions

- Protéger aussi les tags ? Plus tard
- Lien avec la CI
- Petite équipe, grand bénéfice
- Erreur classique
- En vrai
- Scénario avant / après
- Combien de reviews ?
- Bypass et urgence
- Autres branches à protéger ?
- À toi

8. Chapitre 8 - CI légère : des tests automatiques sur la PR

- L'idée sans jargon
- Un exemple mental : site statique + tests JS
- À quoi ça ressemble (idée)
- Faire de la CI un filet, pas une humiliation
- Lien avec les branches protégées
- Que mettre dans une CI légère ?
- Secrets dans la CI
- Erreur classique
- En vrai
- Ce que la CI ne remplace pas
- Faire grandir la CI sans l'exploser
- Échec utile vs échec bruyant
- Qui possède la CI ?
- À toi

9. Chapitre 9 - Tags et releases

- L'idée
- SemVer en version poche
- Release sur GitHub
- Qui crée le tag ?
- Tags et branches
- Lier tag et déploiement
- Erreur classique
- En vrai
- Notes de release qui aident
- Revenir en arrière grâce à un tag
- Tags légers vs annotés
- À toi

10. Chapitre 10 - Cherry-pick : reprendre un commit ailleurs

- L'image
- Un cas concret
- Trouver le commit
- Conflits pendant un cherry-pick
- Plusieurs commits

- Ce n'est pas un remplacement de merge
- Attention a l'historique
- Erreur classique
- En vrai
- Dialogue d'equipe typique
- Dependances entre commits
- Apres le merge du cherry-pick, la feature longue
- A toi

11. Chapitre 11 - Bisect : trouver le commit qui a casse

- L'idee
- Le deroulement
- Automatiser un peu
- Bien choisir good et bad
- Ce que bisect n'est pas
- Limites
- Erreur classique
- En vrai
- Petite histoire
- Conseils pour tester pendant bisect
- Apres avoir trouve
- A toi

12. Chapitre 12 - Mini-projet : simulation d'equipe

- Le scenario
- Preparation
- Role Lea : feature tarifs
- Role Max : fix email
- Role Sam : filets et release
- Option bonus : cherry-pick
- Option bonus : bisect
- Criteres "c'est reussi"
- Erreur classique
- En vrai
- Planning suggere (demi-journee)
- Definition of done du mini-projet
- Piege du perfectionnisme
- A toi

13. Chapitre 13 - A retenir

- Le flux
- Les branches
- Rebase et merge
- Historique
- Revue
- Protection et CI
- Tags et releases

- [Cherry-pick et bisect](#)
- [Contribuer et secrets](#)
- [Phrase DanielCraft](#)
- [A toi](#)

14. [Chapitre 14 - Atelier : flux d'equipe](#)

- [But](#)
- [Materiel](#)
- [Etapas](#)
- [Variante a deux personnes](#)
- [Criteres de reussite](#)
- [Si ca bloque](#)
- [Apres l'atelier](#)

15. [Chapitre 15 - Atelier : revue de code](#)

- [But](#)
- [Preparation](#)
- [Etapas auteur](#)
- [Etapas reviewer](#)
- [Etapas retour auteur](#)
- [Criteres de reussite](#)
- [Pieges a eviter](#)
- [Variante solo](#)
- [Apres l'atelier](#)

16. [Chapitre 16 - Atelier : conflit en equipe](#)

- [But](#)
- [Scenario](#)
- [Etapas](#)
- [Variante rebase](#)
- [Criteres de reussite](#)
- [Si panique](#)
- [Apres l'atelier](#)

17. [Chapitre 17 - Fork et upstream : contribuer ailleurs](#)

- [Les mots](#)
- [Scenario concret](#)
- [Les commandes d'installation](#)
- [Rester a jour avec upstream](#)
- [Le flux de contribution](#)
- [Bonnes manieres](#)
- [Et dans une entreprise ?](#)
- [Erreur classique](#)
- [En vrai](#)
- [PR cross-fork : ce que voit le mainteneur](#)
- [Fork pour experimenter](#)
- [Droits et organisations](#)

- [A toi](#)

18. [Chapitre 18 - Hygiene des secrets : jamais de cles dans Git](#)

- [Ce qu'on ne commit pas](#)
- [.gitignore est ton ami](#)
- [Comment l'equipe partage les secrets](#)
- [Si le mal est fait](#)
- [Revue et secrets](#)
- [Exemple Max](#)
- [Erreur classique](#)
- [En vrai](#)
- [Checklist avant le premier push d'une feature API](#)
- [.env.example est un contrat](#)
- [Secrets et captures d'ecran](#)
- [A toi](#)

19. [Chapitre 19 - Bonnes pratiques d'equipe](#)

- [Ecrire le contrat léger](#)
- [Petites PR, souvent](#)
- [Synchroniser tot](#)
- [Protéger ce qui compte](#)
- [Review comme un collègue, pas comme un juge](#)
- [Noms clairs partout](#)
- [Une intention à la fois](#)
- [Sabbat du force-push](#)
- [Documenter les incidents sans humiliation](#)
- [Chez DanielCraft](#)
- [Erreur classique](#)
- [En vrai](#)
- [Rituels hebdomadaires légers](#)
- [Onboarding d'un nouveau](#)
- [Quand casser une règle](#)
- [Mesure simple](#)
- [A toi](#)

20. [Quiz final](#)

- [Questions](#)
- [Corrigés](#)

21. [Chapitre 21 - Bravo](#)

- [Ce que tu peux faire maintenant](#)
- [La suite](#)
- [Une dernière phrase](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : flux d'équipe, revue, CI, releases.

Tu as déjà fait les bases. Init, clone, status, add, commit, log, branch, merge, pull, push, stash, undo, et une première pull request. Si un détail est flou, ce n'est pas grave : on ne va pas tout refaire. Ce livre part du moment où Git devient un outil d'équipe.

Chez DanielCraft, on aime bien une image simple. Les bases, c'était apprendre à conduire seul sur un parking. Ici, c'est la route à plusieurs : se synchroniser, se revoir le code, protéger la branche principale, et sortir une version sans panique.

Ce que tu gardes en tête

Tu sais créer un commit. Tu sais créer une branche et fusionner. Tu sais pousser et tirer depuis GitHub (ou un équivalent). Tu as vu un conflit une fois. C'est assez pour avancer.

On ne revient pas sur `git init`, ni sur "c'est quoi un commit", ni sur l'installation. Si tu bloques sur ces mots, relis le premier livre Git. Ici, on monte d'un cran : travailler à 2, 3 ou 4 personnes sans se marcher dessus.

La carte du livre

Imagine une petite équipe qui livre un site. Pour que ça marche vraiment, tu as besoin de plusieurs pièces.

D'abord un flux clair : qui pousse ou, quand on tire, comment on se parle. Ensuite une stratégie de branches simple (souvent feature vers main). Rebase et merge, sans dogme : l'idée, les risques, le moment. Un historique propre (messages, petits commits, idée de squash). La revue de code humaine, bienveillante. Les branches protégées. Une CI légère sur les pull requests. Les tags et releases. Puis cherry-pick et bisect, deux outils pour "reprendre un commit" et "trouver le commit casse".

Plus loin : un mini-projet qui simule l'équipe, un recap, trois ateliers, le fork et l'upstream pour contribuer, l'hygiène des secrets, les bonnes pratiques, un quiz, et un bravo.

Un fil rouge

On va souvent parler de trois personnages. Lea (front), Max (back), Sam (qui touche un peu à tout). Ils sont 3 sur un petit site vitrine + formulaire de contact. Parfois on ajoute un bug urgent ou une release. L'idée : des situations concrètes, pas un manuel militaire.

Tu verras le meme geste sous plusieurs angles : partir d'une branche a jour, ouvrir une PR propre, repondre a une review, et ne jamais pousser directement sur `main` sans filet.

Ce dont tu as besoin

Git installe. Un compte GitHub (ou GitLab). Un depot de test (cree-en un vide, pas ton vrai projet client). Un terminal. Et idealement un second compte ou un ami pour jouer la revue. Sinon, tu peux simuler seul en changeant de branche et en te commentant toi-meme.

Comment lire ce livre

Lis dans l'ordre au debut. Flux, branches, rebase vs merge, historique, revue, protection : ca s'enchaîne. Ensuite CI, tags, cherry-pick, bisect. Le mini-projet colle les briques. Les ateliers sont la pour faire, pas seulement lire.

Tu n'as pas besoin d'etre expert DevOps. On reste sur ce qu'une petite equipe utilise vraiment chaque semaine.

Erreur classique

Croire que "je connais Git" = "je sais collaborer". Les bases sont le moteur. L'equipe, c'est le code de la route : priorites, reviews, branches protegees. Sans ca, tout le monde pousse sur `main` et un jour quelqu'un casse la prod un vendredi soir.

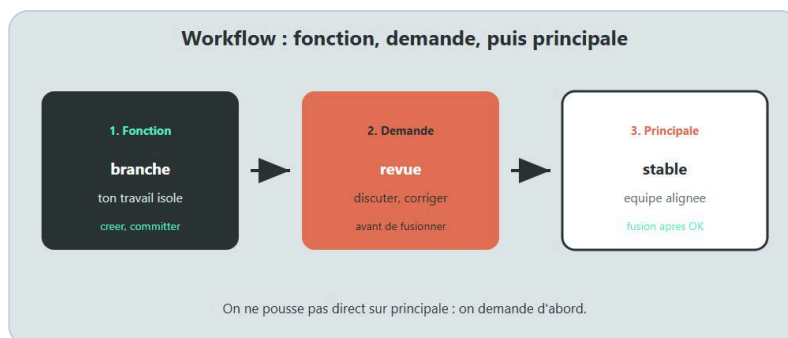
En vrai

Ouvre un vieux depot ou tu as travaille a plusieurs. Note ce qui a frotte : conflits, messages flous, PR trop grosses, `main` non protegee, un secret qui a failli partir. Ce livre repond a ca.

A toi

Ecris en trois phrases la regle d'or que tu voudrais dans ton equipe. Exemple : "jamais de push direct sur main", "toute feature passe par une PR", "on tire main avant de creer une branche". Garde ce but. On y reviendra.

Chapitre 2 - Flux d'equipe : qui pousse ou, quand tirer



Pull, branche, commit, push, revue : le rythme d'equipe.

Git seul, c'est un carnet. Git en equipe, c'est un carnet partage ou tout le monde ecrit en meme temps. Sans accord, c'est le chaos poli : "j'ai pousse sur main", "j'ai ecrase ta branche", "je n'ai pas tire depuis trois jours".

Ce chapitre pose le flux. Pas la theorie des remotes. Le rythme du travail a 2 ou 3 personnes sur un meme site.

L'image du tableau blanc

Chez DanielCraft, on imagine souvent un tableau blanc au milieu de la piece. `main` sur le serveur, c'est le tableau officiel. Ta machine a une copie. La machine de Lea aussi. Si chacun dessine sans regarder le tableau, vous vous ecrasez.

Le flux, c'est juste la regle : je mets a jour ma copie, je dessine sur mon coin (ma branche), je propose mon dessin (PR), quelqu'un regarde, puis on colle sur le tableau.

Une journee type a trois

Le matin, Max arrive. Il veut corriger un bug sur le formulaire. Il fait ceci, dans cet esprit :

```
git switch main
git pull
git switch -c fix/formulaire-email
```

Il travaille. Il commit. Il pousse sa branche, pas `main` :

```
git push -u origin fix/formulaire-email
```

Il ouvre une pull request. Lea regarde. Sam teste le formulaire. On merge. Max revient sur `main` et tire :

```
git switch main
git pull
```

Lea, pendant ce temps, travaille sur `feature/page-tarifs`. Elle a tire `main` ce matin. Elle n'a pas besoin d'attendre Max pour coder. Elle aura besoin de se resynchroniser avant de merger, surtout si Max a touche des fichiers proches.

Qui pousse ou ?

En petite equipe, la regle simple est :

Chacun pousse sur sa branche feature ou fix. Personne ne pousse directement sur `main` (sauf urgence vraiment discutable, et encore : mieux vaut une PR rapide). Le depot distant (`origin`) est la memoire commune. Ta machine n'est pas la source de verite pour les autres.

Si Max commit seulement en local et part en vacances, Lea ne voit rien. Si Max pousse sa branche, Lea peut regarder, tester, commenter.

Push tot, push souvent sur ta branche. Pas sur `main`. Sur ta branche.

Quand tirer (pull) ?

Tire `main` au debut de ta session. Tire `main` avant d'ouvrir une PR. Tire `main` si quelqu'un vient de merger quelque chose d'important pendant que tu codes.

Tu n'as pas besoin de tirer toutes les cinq minutes. Tu as besoin de tirer avant les moments ou ca compte : demarrer, synchroniser, livrer.

```
git switch main
git pull
git switch feature/page-tarifs
git merge main
```

Ici, Lea ramene les nouveautes de `main` dans sa feature. Elle peut aussi rebase (chapitre 4). L'intention est la meme : ne pas decouvrir le conflit le jour du merge final.

Communication minimale

Git ne remplace pas un message. "Je touche `contact.html` aujourd'hui" evite deux personnes sur le meme fichier. "PR prete, besoin d'un regard" evite que la PR pourrisse trois jours.

Dans une equipe de 3, un canal (Discord, Mattermost, salon Slack) suffit. Pas besoin d'un process de 12 pages. Besoin de dire ce qu'on fait.

Le rythme "petit et frequent"

Une feature qui vit deux jours, avec 4 petits commits et une PR de 120 lignes, se review facilement. Une branche qui vit trois semaines, avec 40 commits melanges et 2000 lignes, devient un roman. Personne ne le lit vraiment. On clique Approve en esperant.

Le flux d'equipe marche mieux avec des tranches courtes. Decoupe. Livre. Recommence.

Que faire si tu es bloque par quelqu'un ?

Sam attend la PR de Max pour avancer. Max est en reunion. Options : Sam continue sur une autre tache, ou Sam et Max se mettent d'accord pour une branche partagee (plus rare, plus de communication). Evite de "emprunter" la branche de Max pour y pousser sans prevenir. Surprise = conflits + egos.

Le remote n'est pas magique

`git pull` ramene ce qui est déjà poussé. Si Lea n'a pas poussé, tu ne vois pas son travail. Si tu n'as pas poussé, personne ne voit le tien. Le flux suppose que le travail utile finit sur le serveur, sur une branche claire.

Matin, midi, merge

Le matin : tire `main`, choisis ta tâche, crée ou reprends ta branche. Midi (ou en fin de tranche) : pousse ta branche pour ne pas garder le travail seul sur ton disque. Avant la pause longue : status propre ou stash conscient. En fin de feature : PR + message dans le canal. Après merge : retour sur `main`, pull, suppression de branche.

Ce n'est pas du militaire. C'est un rythme qui évite les "où est passé mon travail ?" et les "je n'avais pas la dernière version".

Fichiers chauds

`index.html`, la config, le schéma de base : si deux personnes y touchent le même jour, parlez-vous. Git fusionnera peut-être. Le sens métier, lui, demandera un humain. Un message "je touche contact.html cet après-midi" coûte dix secondes.

Erreur classique

Coder deux jours sur `main` en local, puis faire un gros push. Ou ne jamais tirer et découvrir 15 commits des autres au moment de merger. Ou pousser `--force` sur une branche partagée "parce que ça a marché chez moi".

Autre piège : croire que "on est que deux, on peut tout faire sur main". À deux, un vendredi soir, un revert malheureux, et vous avez le même problème qu'à dix. La discipline légère protège aussi les petites équipes.

En vrai

Demain matin, avant de coder, fais le rituel :

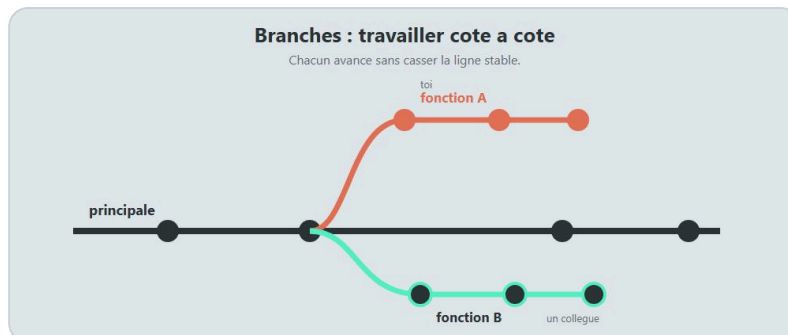
```
git switch main
git pull
git status
```

Puis crée ou reprends ta branche. Note combien de fois cette semaine tu as tiré `main`. Si la réponse est "une fois lundi", augmente le rythme.

À toi

Écris le flux de ton équipe en cinq lignes maximum, comme une recette. Exemple : "1) pull main 2) branche 3) commits 4) push branche 5) PR 6) review 7) merge 8) pull main". Colle-le dans le README du dépôt de test. Tout le monde doit pouvoir le lire en trente secondes.

Chapitre 3 - Strategie de branches simple (main + feature)



main stable + branches de travail courtes.

Une branche, tu la connais deja : une ligne d'historique parallele. En equipe, la question n'est plus "comment creer une branche", c'est "quelles branches on garde, et pour quoi faire".

Si tout le monde invente ses noms et ses durees, le depot devient un grenier. Si vous vous mettez d'accord une fois, Git devient invisible : il porte le travail sans drama.

Le modele simple : main + features

`main` (parfois encore `master` ; on prefere `main`) est la branche de reference. Elle doit etre deployable, ou au moins "connue bonne".

Chaque nouvelle idee vit sur une branche feature (ou fix) :

```
git switch main
git pull
git switch -c feature/page-prix
```

Quand c'est pret et review : merge dans `main` via pull request. Puis on efface la branche feature. Elle a servi. Elle n'a pas vocation a vivre des mois.

C'est le modele qu'on recommande pour 2 a 5 personnes chez DanielCraft. Clair. Suffisant. Facile a expliquer a un stagiaire le premier jour. On l'appelle parfois "GitHub Flow" en version legere : une branche courte, une PR, un merge, on recommence.

Noms de branches qui aident

Un bon nom dit l'intention. `feature/page-prix`, `fix/login-timeout`, `chore/maj-dependances`.

Tu peux utiliser `feature/` pour une nouveaute, `fix/` pour un correctif, `chore/` pour du menage (deps, config), `docs/` pour la doc. Ce n'est pas une loi universelle. C'est une convention d'equipe. Choisissez-en une et tenez-la.

Evite `max-wip`, `test2`, `final-final-v3`. Dans six mois, personne ne saura ce que c'etait. Y compris Max.

Combien de temps une branche vit-elle ?

Idealement : heures ou quelques jours. Pas trois semaines sans sync.

Plus une branche vit longtemps, plus `main` avance sans elle, plus le retour sera douloureux. Synchronise souvent :

```
git switch main
git pull
git switch feature/page-prix
git merge main
```

Ou avec rebase (chapitre suivant). L'idée : ramener regulierement les nouveautes de `main` dans ta feature, avant le gros conflit du vendredi.

Et develop ? Et release ?

GitFlow (avec `develop`, `release/`, `hotfix/`) existe. Il aide les grosses organisations avec des calendriers de release lourds. Pour une petite equipe produit qui deploie souvent, c'est souvent trop de ceremony.

Trunk-based development (tout le monde merge tres souvent dans `main`, parfois avec feature flags) est une autre ecole. Puissant, mais demande discipline et souvent une CI solide.

Pour ce livre : maitrise d'abord main + feature + PR. Tu pourras complexifier plus tard si le besoin est reel, pas par snobisme.

Une branche par intention

Si Lea corrige un bug login et ajoute une page prix dans la meme branche, la review melange deux sujets. Au merge, si la page prix est douteuse mais le bug urgent, vous etes bloques.

Deux branches. Deux PR. Le bug peut partir ce soir. La page prix demain apres discussion.

Branches partagees

Parfois deux personnes travaillent sur la meme feature. Possible. Alors : communication forte, petits commits, pull frequents sur la branche feature. Evite que chacun reecrive l'historique de l'autre (rebase force) sans se parler.

En general, preferer decouper le travail pour que chacun ait sa branche. Moins de friction.

Hotfix : le petit frere du fix

Un bug en production. Sam cree `fix/bouton-payer` depuis `main` a jour, corrige, ouvre une PR courte, fait merger vite, deploie. Meme modele que la feature. Juste plus urgent, donc plus petit, plus cible.

Ne cree pas une usine a gaz "hotfix process" a trois personnes. Cree une branche claire et une PR lisible.

Nettoyage

Apres merge, supprime la branche locale et distante. GitHub a souvent une case "Delete branch" apres merge. Localement :

```
git switch main
git pull
git branch -d feature/page-prix
```

De temps en temps, liste les vieilles branches. Un depot avec 80 branches mortes fatigue tout le monde.

```
git branch -a
```

Carte mentale pour un stagiaire

Jour 1, tu peux coller ça au mur : "1. Je tire main. 2. Je cree feature/ou-fix/quelque-chose. 3. Je commit. 4. Je pousse ma branche. 5. J'ouvre une PR. 6. On review. 7. On merge. 8. Je retive main et j'efface ma branche." Si le stagiaire retient ça, il est deja utile a l'equipe.

Erreur classique

Creer `feature/mega-refonte` et y vivre trois mois. Ou committer directement sur `main` "juste pour un petit truc". Les petits trucs s'accumulent. Puis un jour `main` est cassee et personne ne sait quel petit truc a fait le mal.

Autre piege : dix branches `feature/tmp` sans description. Le grenier gagne.

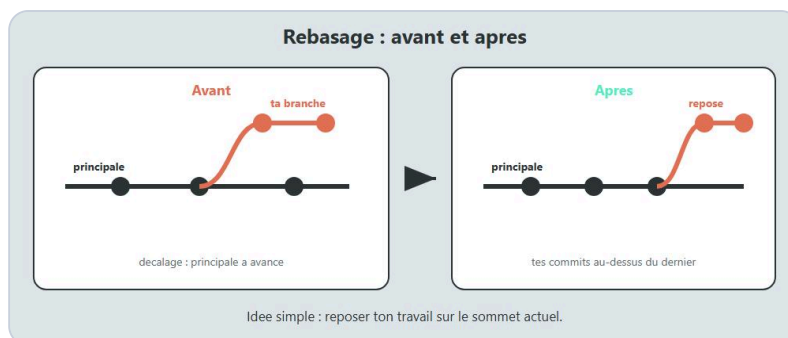
En vrai

Ouvre ton depot. Liste les branches avec `git branch -a`. Combien sont mortes ? Combien ont un nom obscur ? Note trois renommages ou suppressions a faire. Hygiene = clarte.

A toi

Ecris dans le README : "On travaille avec main + feature/*. Toute integration passe par une PR. Les branches feature vivent moins d'une semaine si possible." Adapte si besoin, mais ecris quelque chose. Une regle ecrite bat une regle imaginee.

Chapitre 4 - Rebase vs merge : idee, quand, risques



Rebase ou merge : deux facons de rejoindre l'histoire.

Tu as deja fusionne avec `git merge`. Ca cree parfois un commit de merge, et l'historique montre que deux lignes se sont rejointes. Le rebase, c'est une autre facon de "mettre a jour" ta branche : rejouer tes commits comme s'ils etaient partis plus tard, sur une base plus recente.

Ce n'est pas une religion. Chez DanielCraft, on refuse le dogme "toujours rebase" ou "jamais rebase". On veut que tu comprennes l'idee, le moment, et le danger.

L'idee en une image

Imagine que `main` avance. Ta feature est partie d'un vieux point. Merge, c'est coller les deux histoires avec un noeud. Rebase, c'est decouper tes commits de ta feature et les recoller proprement au bout de `main`, un par un.

Resultat frequent du rebase : un historique plus lineaire, plus facile a lire avec `git log --oneline`.

Resultat frequent du merge : un historique qui montre vraiment les allers-retours d'equipe.

Les deux sont valides. Le choix depend du contexte et de l'accord d'equipe.

Un cas concret : Lea met a jour sa feature

Lea est sur `feature/page-tarifs`. Max a merge un fix sur `main`. Lea veut ces changements avant sa PR.

Option merge :

```
git switch feature/page-tarifs
git fetch origin
git merge origin/main
```

Option rebase :

```
git switch feature/page-tarifs
git fetch origin
git rebase origin/main
```

Dans les deux cas, elle peut avoir des conflits. Elle les regle. La difference, c'est la forme de l'historique ensuite.

Quand le rebase aide

Le rebase aide souvent sur une branche feature personnelle, pas encore partagee (ou partagee seulement par toi), pour la "reposer" sur `main` avant la review. L'historique de la PR devient une suite claire de commits, sans noeud de merge intermediaire.

Il aide aussi pour nettoyer localement avant de pousser (voir chapitre 5 sur l'historique propre), parfois avec un rebase interactif. Attention : interactif = plus de puissance, plus de responsabilite.

Quand le merge est plus simple (et plus sur)

Si plusieurs personnes poussent sur la meme branche feature, un rebase + force push peut casser le travail des autres. Merge est alors plus doux : il n'exige pas de reecrire l'historique deja publie.

Si tu n'es pas a l'aise, merge. Un merge un peu bruyant bat un rebase mal compris qui reecrit l'histoire de tout le monde.

Le risque numero un : reecrire l'histoire partagee

Rebase change les identifiants de commits (les hash). Ce ne sont plus "les memes" commits, ce sont des copies rejouees. Si tu as deja pousse ces commits et que d'autres les ont tires, tu dois souvent pousser avec `--force` (ou `--force-with-lease`). Sur `main`, c'est en general interdit et dangereux. Sur une branche perso, c'est parfois OK si tu es seul dessus.

Regle d'or : ne rebase pas `main`. Ne force-push pas `main`. Ne rebase pas la branche d'un collegue sans qu'il le sache.

```
# Sur TA feature, apres rebase local, si deja poussee :  
git push --force-with-lease
```

`--force-with-lease` est plus poli que `--force` : il refuse d'ecraser si quelqu'un a pousse entre-temps. Prefere-le.

Conflits : meme stress, meme methode

Que tu merges ou rebases, un conflit veut dire : deux changements touchent la meme zone. Tu ouvres le fichier, tu choisis (ou combines), tu marques resolu.

En merge :

```
git add fichier.html  
git merge --continue
```

En rebase (souvent) :

```
git add fichier.html  
git rebase --continue
```

Si tu es perdu pendant un rebase :

```
git rebase --abort
```

Tu reviens a l'etat d'avant le rebase. Respire. Recommence autrement si besoin (par exemple avec un merge).

Sans dogme : une politique d'equipe possible

Exemple de politique simple pour une equipe de 3 :

On merge les pull requests dans `main` (bouton Merge sur GitHub, parfois "squash merge" - chapitre 5). Sur sa feature perso, chacun peut rebase sur `main` avant la PR pour limiter le bruit. Personne ne rebase une branche partagee sans message dans le canal.

Ecrivez deux phrases dans le README. Ca suffit.

Erreur classique

Faire un rebase sur `main` local puis pousser en force "parce que mon log est plus joli". Ou lancer un rebase interactif le jour de la release sans filet. Ou expliquer a un stagiaire que merge est "moche" : tu crees de la honte inutile. L'outil sert l'equipe, pas l'inverse.

En vrai

Sur un depot de test, cree une branche, fais deux commits, avance `main` avec un autre commit, puis teste les deux options (merge puis, dans une autre copie de branche, rebase). Regarde `git log --oneline --graph`. Vois la difference avec tes yeux, pas seulement dans un article.

```
git log --oneline --graph --decorate -15
```

A toi

Choisis avec ton equipe (meme si l'equipe c'est toi et un ami) : "rebase OK sur feature perso, merge pour integrer dans main" ou "merge partout pour simplifier". Ecris le choix. Le pire choix, c'est le non-choix ou chacun fait sa religion en silence.

Chapitre 5 - Historique propre : messages, petits commits, squash

L'historique Git, c'est le journal de bord du projet. Dans six mois, Lea ouvrira `git log` pour comprendre pourquoi le formulaire valide l'email d'une certaine façon. Si elle lit "fix", "wip", "asdf", "final", elle perdra du temps. Si elle lit des messages clairs et des commits digérés, elle avancera.

Un historique propre n'est pas de la vanité. C'est de la gentillesse envers le futur (toi y compris).

Des messages qui racontent le pourquoi

Un bon message dit surtout l'intention. "Corrige le timeout du login" bat "modif fichier". "Ajoute la section tarifs sur la page offres" bat "update".

Tu n'as pas besoin d'un roman. Une ligne claire suffit souvent. Si le changement est subtil, une deuxième ligne après une ligne vide aide.

```
Corrige la validation email du formulaire contact

Le champ acceptait les adresses sans point dans le domaine.
```

Chez DanielCraft, on préfère un ton simple, factuel, sans emoji obligatoire, sans roman policier.

Petits commits, grandes PR ? Non : petits commits et PR raisonnables

Un commit = une intention cohérente. Tu peux avoir plusieurs commits dans une PR, tant que chaque commit raconte une étape logique : structure HTML, puis styles, puis texte. Évite le commit unique de 80 fichiers "toute la feature" si tu peux découper sans souffrir.

Inversement, évite 40 commits "typo", "typo2", "aaa", "wip" laissés tels quels sur `main`. Avant la merge, tu peux nettoyer (squash, ou rebase interactif si tu maîtrises).

L'idée du squash

Squash = compresser plusieurs commits en un (ou en peu). Sur GitHub, l'option "Squash and merge" prend tous les commits de la PR et les intègre dans `main` comme un seul commit. Pratique pour garder `main` lisible quand la feature a vécu avec des allers-retours.

Ce n'est pas obligatoire. Certaines équipes aiment garder tous les commits de la PR. D'autres squashtent toujours. Choisissez. L'important est que `main` reste navigable.

Amend : corriger le tout dernier commit (avec prudence)

Tu viens de commit et tu as oublié un fichier, ou une typo dans le message. Si tu n'as pas encore poussé (ou si tu es seul sur la branche et d'accord pour réécrire) :

```
git add fichier-oublie.css
git commit --amend --no-edit
```

`--no-edit` garde le même message. Sans ce flag, tu peux retoucher le message.

N'amends pas un commit déjà sur `main` partagé. N'amends pas le travail d'un autre. Amend = micro-rewrite. Puissant, local, dangereux si mal placé.

Ce qu'on ne met pas dans l'historique

Les mots de passe. Les clés API. Les dumps de base. Les fichiers `.env` (chapitre 18). Un historique "propre" commence aussi par ne pas y coller des secrets. Une fois poussé, un secret est difficile à vraiment effacer des copies et des forks.

Relire avant de demander la review

Avant d'ouvrir la PR (ou juste après), regarde :

```
git log main..HEAD --oneline
git diff main...HEAD
```

Est-ce que la liste des commits raconte une histoire ? Est-ce qu'il y a un fichier de debug à retirer ? Un `console.log` oublié ? Un commentaire "TODO enlever" qui devait partir ?

Cinq minutes ici épargnent vingt minutes de review grevée.

Conventional commits ? Optionnel

Tu verras parfois `feat:`, `fix:`, `docs:`. Utile si l'équipe s'en sert pour des changelogs automatiques. Pas obligatoire pour bien travailler. Mieux vaut un message clair sans préfixe qu'un préfixe vide de sens (`feat: fix stuff`).

Si vous adoptez une convention, gardez-la légère et documentée en cinq lignes.

Erreur classique

Honte du "mauvais historique" au point de ne plus oser commit. Commit souvent en local sur ta feature. Nettoie avant d'intégrer si besoin. L'inverse aussi : pousser un journal de bord illisible sur `main` parce que "on verra plus tard". Plus tard, c'est Lea un mardi à 18h.

En vrai

Prends une ancienne PR (ou un vieux `git log`). Note trois messages obscurs. Réécris-les comme tu aurais voulu les lire. Cet exercice calibre ton prochain commit.

Un après-midi chez Lea, Max et Sam

Lea a six commits sur sa feature tarifs : "wip", "wip2", "styles", "fix typo", "ok", "ok final". Avant d'ouvrir la PR, elle regarde le log et se dit que Sam va souffrir. Elle décide de squash via le bouton GitHub au merge, et elle réécrit le message final : "Ajoute la section tarifs sur la page offres". `main` reste lisible. Sa branche feature peut garder ses allers-retours : ce n'est qu'un brouillon.

Max, lui, préfère des commits déjà propres avant la PR. Les deux approches marchent si l'équipe est d'accord sur ce que doit ressembler `main`.

Relire le diff comme un inconnu

Imagine que tu ne connais pas le projet. Est-ce que le message + le diff racontent une histoire ? Si tu dois expliquer oralement pendant cinq minutes pour que le commit ait un sens, le message est trop maigre. Ajoute une ligne. Pas un roman : une intention.

Fichiers qui ne devraient pas être là

Un `debug.log`, un `node_modules` oublié, un zip, une image de 12 Mo "temporaire". L'historique propre, c'est aussi le périmètre des fichiers. `.gitignore` aide. La revue aussi. Un commit "supprime `node_modules`" après coup reste un commit gênant dans le log.

Quand WIP est acceptable

Sur ta branche perso non partagée, "wip" en local le temps d'un café, pourquoi pas. Avant la review, nettoie ou squash. Laisser "wip" sur `main` après merge, c'est offrir une énigme à toute l'équipe.

A toi

Pour ta prochaine branche, impose-toi cette règle : chaque message répond à "pourquoi ce changement existe". Pas "quels fichiers". Le pourquoi. Puis ouvre le log et lis-le à voix haute. Si tu souris nerveusement, recommence le message.

Chapitre 6 - Revue de code humaine

La pull request n'est pas un portail magique. C'est une conversation. Quelqu'un propose un changement. Quelqu'un d'autre regarde, pose des questions, suggere, approuve. Git transporte les diffs. Les humains portent le sens.

Sans revue, on merge des bombes polies. Avec une revue toxique, on merge moins, on cache, on a peur. L'objectif : une revue utile et bienveillante.

Pourquoi reviewer ?

Parce que quatre yeux voient plus que deux. Parce que Max connaît le back et peut voir que le front de Lea casse une API. Parce que Sam se souvient qu'un cas mobile n'est pas testé. Parce que écrire pour être lu oblige à clarifier.

Chez DanielCraft, la revue est un filet, pas un tribunal.

Quoi regarder (sans tout relire ligne à ligne)

Tu n'as pas à être un compilateur humain. Priorise.

Est-ce que la PR fait ce qu'elle dit ? Est-ce que le périmètre est raisonnable (pas 30 sujets) ? Est-ce qu'il y a un risque sécurité évident (secret, injection grossière, permission trop large) ? Est-ce que les noms aident ? Est-ce que tu saurais maintenir ce code dans trois mois ? Est-ce que l'auteur a indiqué comment tester ?

Regarde aussi ce qui manque : gestion d'erreur, cas vide, accessibilité d'un bouton, message utilisateur incompréhensible.

Tu peux laisser passer un détail de style mineur si l'équipe n'a pas de règle stricte. Tu ne laisses pas passer un formulaire qui envoie des données en clair par erreur.

Le ton qui aide

Préfère "Ici, si l'email est vide, on n'affiche pas d'erreur - on pourrait bloquer l'envoi" à "C'est nul". Préfère "Je ne comprends pas cette condition, tu peux m'expliquer ?" à "Lol quoi". Préfère "Suggestion :" à "Obligatoire :" quand c'est vraiment une préférence.

Dis aussi ce qui est bien. "Clair la séparation HTML / CSS" prend trois secondes et donne de l'énergie. La revue n'est pas seulement une chasse aux défauts.

En tant qu'auteur de la PR

Aide le reviewer. Titre clair. Description courte : but, changements principaux, comment tester. Capture d'écran si c'est visuel. Lien vers une issue si elle existe.

```
## But
Afficher les tarifs sur /offres

## Changements
- section tarifs dans offres.html
- styles associes
- liens depuis l'accueil

## Comment tester
1. Ouvrir /offres
2. Verifier mobile
3. Cliquer "Contact" depuis un tarif
```

Reponds aux commentaires sans te defendre par principe. Si tu n'es pas d'accord, explique calmement. Si tu corriges, pousse un commit et dis "c'est a jour".

Taille de PR

Une PR de 150 lignes se review. Une PR de 2500 lignes se feint. Decoupe. Si tu ne peux pas decouper (trop lie), guide le reviewer : "commence par `api.js`, puis le template".

Temps de reponse

Une PR qui attend une semaine pourrit. L'auteur rebase, le contexte se perd, la motivation tombe. En petite equipe, vise un premier regard en 24h quand c'est possible. Meme un "je regarde demain matin" compte.

Approve, request changes, commentaire

Sur GitHub, tu peux commenter sans bloquer, demander des changements, ou approuver. Utilise "request changes" pour les vrais blocants. N'approuve pas par pitie si tu n'as pas regarde. N'utilise pas "request changes" pour une virgule.

Erreur classique

La revue ego : "moi j'aurais fait autrement" sur chaque ligne. Ou la revue fantome : Approve sans ouvrir les fichiers. Ou l'auteur invisible qui n'ecrit aucune description. Les trois cassent le jeu.

En vrai

Prends une PR passee (la tienne ou une publique). Ecris trois commentaires bienveillants et utiles que tu aurais pu laisser. Puis ecris le commentaire toxique equivalent... et jette-le. Sens la difference. Garde la premiere version comme modele mental.

Exemple de fil de review (ton juste)

Sam sur la PR de Lea :

"J'ai suivi tes etapes de test sur mobile : le bloc tarifs passe bien. Une question : sur tres petit ecran, le prix passe sous le bouton - voulu ? Si non, on peut empiler en colonne des 480px. Suggestion, pas bloqueur si tu vises desktop d'abord."

Lea repond : "Pas voulu. Je pousse un correctif ce soir." Elle pousse. Sam Approuve. Merge. Quatre messages, zero ego, un meilleur site.

Ce qu'on ne review pas (ou peu)

Le gout personnel sans regle d'equipe ("moi j'aime les simples quotes"). La refonte totale hors sujet. Le "j'aurais tout ecrit autrement" sans proposition actionnable. La revue n'est pas un concours de style.

Check-list mentale du reviewer (sans en faire une religion)

Je comprends le but. J'ai regarde les fichiers touches. J'ai teste ou j'ai dit pourquoi je ne pouvais pas. J'ai signale les risques. J'ai laisse au moins une remarque utile ou un approve conscient. C'est deja une bonne review.

Quand tu n'es pas expert du domaine

Tu peux quand meme aider : clarte des noms, cas limites evidents, secrets, "je ne comprends pas ce bloc". Dire "je ne connais pas bien cette API, Max peut-tu jeter un oeil ?" est professionnel. Ce n'est pas un echec.

A toi

Ajoute dans le README de l'equipe trois phrases : "On review pour aider.", "On explique le comment tester.", "On dit aussi ce qui est clair." Lis-les avant ta prochaine review.

Chapitre 7 - Branches protegees : pas de push direct sur main

Tu peux ecrire la belle regle "on ne pousse pas sur main" dans le README. Un jour, quelqu'un oubliera. Un autre jour, un stagiaire ne l'aura pas lue. Un vendredi, tu pousseras toi-meme "juste un hotfix" a 19h.

Les branches protegees, c'est le filet technique. GitHub (et GitLab, et d'autres) peuvent refuser le push direct sur `main` et imposer le passage par une pull request.

L'idee

Protoger `main`, c'est dire au serveur : "cette branche est speciale". On n'y ecrit pas comme sur un brouillon. On y arrive apres review (et souvent apres des checks automatiques).

Lea ne peut plus faire par erreur :

```
git push origin main
```

si la protection est bien configuree. Elle devra pousser sa branche et ouvrir une PR.

Ou ca se regle (idee GitHub)

Sur GitHub, dans le depot : Settings, puis la zone Branch protection / Rules. Tu choisis la branche `main`. Tu actives des options du genre : exiger une pull request avant merge, exiger un certain nombre d'approbations, exiger que les status checks passent (CI), interdire le force push, interdire la suppression de la branche.

Tu n'as pas besoin de tout cocher le premier jour. Pour une equipe de 3 chez DanielCraft, un bon minimum est : pas de push direct, PR obligatoire, au moins une review, pas de force push sur `main`.

Les ecrans changent selon les versions de GitHub. L'intention reste : regles sur `main`, filet visible.

Ce que ca change au quotidien

Max finit un fix. Il pousse `fix/login`. Il ouvre une PR vers `main`. Lea approuve. Max clique Merge. `main` avance. Le depot a refuse les raccourcis.

Si quelqu'un tente un push direct, le serveur dit non. C'est sec. C'est bon. Mieux vaut un message d'erreur clair qu'une prod cassee.

Administrateurs et exceptions

Parfois les admins peuvent bypasser. C'est tentant. Utilisez cette exception rarement, et seulement pour des urgences vraiment discutees. Si les admins bypassent tous les jours, vous n'avez plus de protection : vous avez un theatre.

Protoger aussi les tags ? Plus tard

On peut proteger les tags pour eviter qu'on deplace une version `v1.2.0`. Utile quand les releases comptent vraiment. Pour commencer, protege `main`. Les tags viennent au chapitre 9.

Lien avec la CI

Si tu exiges que les checks soient verts avant merge, une CI legere (chapitre 8) devient un garde-fou automatique. Review humaine + tests automatiques = double filet. Ni l'un ni l'autre n'est parfait seul.

Petite equipe, grand benefice

"On est que deux, pas besoin." Justement : a deux, un clic malheureux n'a pas de troisieme personne pour rattraper dans la seconde. La protection est rapide a activer et evite des soirees tristes.

Erreur classique

Activer la protection sans prevenir l'equipe : tout le monde panique ("Git est casse"). Ou activer vingt regles d'un coup (3 reviews, 12 checks, signatures) au point de bloquer tout livraison. Commence simple. Explique. Ajuste.

Autre piege : proteger `main` mais laisser `master` ou une vieille branche de prod non protegee. Verifie quelle branche est vraiment deployee.

En vrai

Sur un depot de test (pas le client critique), active une regle : PR obligatoire sur `main`. Demande a un collegue (ou a ton second compte) d'essayer un push direct. Voyez le refus. Puis faites le chemin PR. Le corps retient mieux qu'un paragraphe.

Scenario avant / apres

Avant : Max pousse sur `main` un hotfix a 19h12. Le site casse. Lea tire `main` le lendemain et passe une heure a comprendre. Personne n'a review. Aucune CI.

Apres : Max pousse `fix/...`, ouvre une PR, Sam regarde deux minutes, CI verte, merge. Meme urgence relative, mais un filet. Si la CI est rouge, Max corrige avant d'infecter `main`.

Combien de reviews ?

A trois personnes, une approval suffit souvent. A dix, parfois deux. N'impose pas trois approvals si vous etes trois et que deux sont en conges : vous vous bloquez. Adaptez la regle a la taille reelle.

Bypass et urgence

Vraie urgence prod : parfois un admin bypass, merge, deploye, puis ouvre une issue "post-mortem leger" le lendemain. Fausse urgence ("j'ai la flemme de la PR") : non. L'equipe doit sentir la difference.

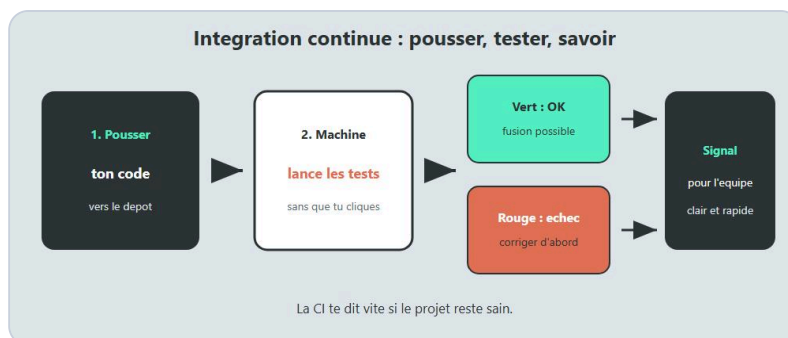
Autres branches a proteger ?

Parfois `production` ou `release/*`. Pour ce livre et une petite equipe : protegez la branche que tu deployes vraiment. Souvent `main`. Une seule source de verite.

A toi

Note dans le README : "main est protegee : pas de push direct, PR + 1 review minimum." Ajoute le lien vers les settings si utile. La regle ecrite + la regle technique = coherence.

Chapitre 8 - CI legere : des tests automatiques sur la PR



La CI verifie la PR avant de fusionner.

La revue humaine regarde le sens. La CI regarde la mecanique : est-ce que ca build ? Est-ce que les tests passent ? Est-ce que le linter crie ? Tu n'as pas besoin d'une usine NASA. Tu as besoin d'un feu vert / feu rouge simple sur chaque pull request.

CI veut dire Continuous Integration : a chaque proposition de changement, une machine rejoue des verifications.

L'idee sans jargon

Lea ouvre une PR. GitHub Actions (ou GitLab CI, ou autre) lance un petit script : installer, tester, parfois builder. Au bout de deux minutes, un point vert ou rouge apparait sur la PR. Max voit le rouge avant de merger. Lea corrige. On gagne une categorie entiere de "ouille, j'avais oublie".

Chez DanielCraft, on aime la CI qui tient sur une page de config et qui finit vite. Si la CI met 40 minutes, les gens la contournent mentalement.

Un exemple mental : site statique + tests JS

Imagine le site de Lea, Max et Sam. Ils ont quelques tests automatiques sur des fonctions de validation (email, telephone). Sur chaque PR, la CI fait :

1. Recuperer le code de la PR
2. Installer les dependances
3. Lancer les tests
4. Afficher succes ou echec

Pas besoin de deployer en production depuis la CI le premier jour. Juste verifier.

A quoi ca ressemble (idee)

Sur GitHub Actions, un fichier dans `.github/workflows/ci.yml` decrit le travail. L'idee generale, pas un tutoriel exhaustif :

```
on: pull_request
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - recuperer le code
      - installer Node (ou Python, etc.)
      - npm test (ou pytest, etc.)
```

Tu adapteras a ton stack. L'important est le reflexe : PR = verification automatique.

Faire de la CI un filet, pas une humiliation

Quand c'est rouge, le message doit aider. "3 tests failed" avec le nom du test bat "Error". L'auteur corrige sans honte. La CI n'est pas la, pour punir. Elle est la pour attraper tot.

Si la CI est flaky (rouge au hasard), corrige-la vite. Une CI menteuse est pire que pas de CI : on ignore le rouge.

Lien avec les branches protegees

Tu peux exiger que le check `test` soit vert avant merge. Alors plus personne ne merge "on verra bien". Le bouton Merge attend le vert.

Que mettre dans une CI legere ?

Pour commencer : installer + tests unitaires rapides. Ensuite eventuellement lint. Ensuite eventuellement un build. Evite d'ajouter dix outils le meme jour. Chaque outil doit gagner sa place.

Si vous n'avez aucun test encore, une CI qui lance au moins un script `npm test` (meme minimal) cree le rituel. Puis vous ajoutez de vrais tests au fil des bugs.

Secrets dans la CI

Parfois la CI a besoin d'un token. Stocke-le dans les secrets du depot (reglages GitHub), pas dans le fichier yaml. Le chapitre 18 insiste : les cles ne vivent pas dans Git.

Erreur classique

Copier une mega config internet avec cache, matrix, deploy, notifications Slack, et ne plus comprendre pourquoi c'est rouge. Ou bloquer le merge sur un check optionnel qui casse souvent. Ou n'avoir de CI que sur `main` apres merge : trop tard, le mal est deja integre.

En vrai

Ajoute un test minuscule qui echoue si une fonction `validerEmail` accepte `"a@b"`. Ouvre une PR qui casse volontairement. Regarde le rouge. Corrige. Regarde le vert. Ce petit theatre ancre le geste.

Ce que la CI ne remplace pas

Elle ne remplace pas la revue humaine sur le sens produit. Elle ne garantit pas qu'un bouton est joli. Elle ne comprend pas qu'un texte client est faux. Elle attrape les regressions mecaniques. Garde les deux filets.

Faire grandir la CI sans l'exploser

Semaine 1 : tests unitaires. Semaine 3 : lint. Plus tard : build de preview. Chaque ajout doit reduire une douleur réelle. Si personne ne regarde un check, retire-le ou repare-le.

Echec utile vs echec bruyant

Utile : "test validerEmail refuse a@b". Bruyant : stack trace de 200 lignes sans nom de test. Investis cinq minutes pour rendre le rouge lisible. Toute l'equipe gagne a chaque PR.

Qui possede la CI ?

Tout le monde. Si c'est "le truc de Sam", Sam en vacances = CI rouge ignoree. Documente en cinq lignes comment relancer et ou est le fichier de config.

A toi

Ecris la promesse de votre CI en une phrase dans le README : "Chaque PR lance les tests ; on ne merge pas au rouge." Si vous n'avez pas encore de CI, écris la phrase et mets la mise en place dans le mini-projet (chapitre 12) ou juste apres.

Chapitre 9 - Tags et releases

Un commit a un hash du genre `a3f19c2`. Pratique pour Git, penible pour les humains. Un tag, c'est un nom stable colle sur un commit : `v1.0.0`, `v1.1.0`, `v2.0.0`. Une release, c'est souvent ce tag + des notes ("ce qui change pour les utilisateurs").

Quand Sam dit "le bug est sur la 1.2", tout le monde doit pouvoir retrouver le meme code. Les tags servent a ca.

L'idée

Tu livres le site. Ca marche. Tu marques le commit actuel :

```
git tag -a v1.0.0 -m "Premiere version publique du site"
git push origin v1.0.0
```

`-a` cree un tag annote (avec message et auteur). C'est en general preferable aux tags legeres pour les versions.

Plus tard, tu pourras revenir a ce point precis, construire a partir de lui, ou comparer :

```
git checkout v1.0.0
```

(tu seras en "detached HEAD" : normal pour inspecter une version ; reviens sur une branche apres)

```
git switch main
```

SemVer en version poche

Beaucoup d'equipes utilisent Major.Minor.Patch : `v1.2.3`.

Patch : correctif sans changer l'usage (`v1.2.3` -> `v1.2.4`). Minor : nouveaute compatible (`v1.2.4` -> `v1.3.0`). Major : changement qui casse des usages (`v1.3.0` -> `v2.0.0`).

Pour un site vitrine, tu peux simplifier : `v1`, `v2`, ou des dates. L'important est d'etre coherent dans l'equipe.

Release sur GitHub

Sur GitHub, tu peux creer une Release a partir d'un tag : titre, notes, fichiers attaches parfois. Lea ecrit : "Ajout page tarifs, fix formulaire email". Max deploie en sachant quoi tester. Le client comprend ce qui a bouge.

Chez DanielCraft, une release courte et honnete bat un roman marketing.

Qui cree le tag ?

Souvent la personne qui livre, apres le merge sur `main`, sur le commit de `main` a jour. Pas sur une feature random. Pas "je tag chez moi sans pousser" : les autres ne verront rien.

```
git switch main
git pull
git tag -a v1.2.0 -m "Tarifs + fix email"
git push origin v1.2.0
```

Tags et branches

Une branche bouge. Un tag, en principe, non. `main` avance. `v1.2.0` reste le souvenir d'un instant. C'est pour ça que c'est utile : point fixe dans un fleuve.

Evite de déplacer un tag déjà publié (`git tag -f` puis force push) sauf correction immédiate et annoncée. Les gens ont peut-être déjà tiré `v1.2.0`.

Lier tag et déploiement

Idealement : le même commit tagué est celui que tu déploies. Si tu déploies un commit et tu tags un autre, tu mens à ton futur toi. Aligne les gestes : merge, tag, deploy, notes.

Erreur classique

Ne jamais taguer, puis chercher pendant une heure "la version de mardi". Ou taguer `v1` dix fois en le déplaçant. Ou mettre dans les notes de release des secrets (non). Ou oublier de pousser le tag : tout le monde croit que la 1.2 n'existe pas.

En vrai

Sur le dépôt de test, crée `v0.1.0` sur `main`, pousse le tag, crée une Release GitHub avec trois lignes de notes. Ensuite avance `main` d'un commit et crée `v0.1.1`. Liste :

```
git tag
```

Sens la chronologie.

Notes de release qui aident

Mauvais : "divers fixes". Meilleur : "Fix validation email du formulaire contact. Ajout section tarifs sur /offres. Pas de migration base."

Ecris pour la personne qui va tester et pour celle qui va communiquer au client. Pas pour impressionner.

Revenir en arrière grâce à un tag

Un bug grave sur `v1.3.0`. Tu sais que `v1.2.0` était sain. Tu peux inspecter, comparer, ou redéployer l'ancien artefact issu de ce tag selon votre pipeline. Sans tag, tu fouilles les dates et les souvenirs. Avec tag, tu as un nom.

```
git log --oneline v1.2.0..v1.3.0
```

Tu vois ce qui a changé entre les deux. Utile pour enquêter (complément de bisect parfois).

Tags legers vs annotes

Les tags legers sont juste un nom. Les annotes portent un message et des metadonnees. Pour les versions, prefere annote. Pour un marqueur purement local de travail, un leger peut suffire (et souvent tu n'en as pas besoin).

A toi

Decide avec l'equipe le schema de versions (SemVer simple ou autre) et qui a le droit de publier une release. Une phrase dans le README suffit : "On tague main apres chaque livraison utile : vX.Y.Z."

Chapitre 10 - Cherry-pick : reprendre un commit ailleurs

Parfois tu as un commit parfait... mais pas sur la bonne branche. Le fix est sur une feature longue. La prod brule. Tu voudrais juste cette correction sur `main`, sans merger toute la feature.

`git cherry-pick` copie un commit (son diff) et le rejoue sur ta branche actuelle. Nouveau hash, meme idee.

L'image

Chez DanielCraft, on dit : tu cueilles une cerise sur un arbre (une branche) et tu la poses sur un autre plateau. Tu ne demenages pas l'arbre.

Un cas concret

Max a trois commits sur `feature/refonte-login` :

```
a111 - prepare la structure
b222 - corrige le timeout session <-- celui-la est urgent
c333 - nouveau design du formulaire
```

La prod a besoin de `b222` maintenant. Merger toute la feature amenerait le design inacheve. Cherry-pick :

```
git switch main
git pull
git switch -c fix/timeout-session
git cherry-pick b222
```

Si ca s'applique proprement, tu as le fix sur une branche courte. Tu ouvres une PR. Tu merges. La feature de Max continue sa vie. Plus tard, quand la feature rejoindra `main`, Git gerera souvent le doublon assez bien (pas toujours magique : reste attentif).

Trouver le commit

```
git log --oneline feature/refonte-login
```

Tu copies le hash court ou long. Tu cherry-pick ce hash.

Conflits pendant un cherry-pick

Comme un merge ou un rebase, ca peut conflictuer. Tu resous, tu `git add`, tu continues :

```
git cherry-pick --continue
```

Ou tu abandonnes :

```
git cherry-pick --abort
```

Plusieurs commits

Tu peux enchaîner plusieurs cherry-picks, ou donner un intervalle selon les cas. Pour débiter, un commit à la fois. C'est plus clair mentalement.

Ce n'est pas un remplacement de merge

Si tu cherry-picks vingt commits d'une feature, tu es en train de merger à la main, mal. Cherry-pick brille pour 1 (parfois 2-3) commits cibles. Pour une feature entière : merge ou PR normale.

Attention à l'historique

Après cherry-pick, tu as deux commits cousins (même changement, hash différents) sur deux lignes d'histoire. Les messages restent importants : garde un message qui dit encore le pourquoi. Si besoin, retouche le message au moment du pick (options avancées) ou amend local avant push.

Erreur classique

Cherry-pick depuis le mauvais hash. Ou cherry-pick d'un commit qui dépend d'un commit précédent non pris (le code ne compile plus). Ou cherry-pick directement sur `main` sans PR alors que `main` est protégée... passe par une branche fix.

Autre piège : utiliser cherry-pick pour "éviter la review". Non. Tu ouvres quand même une PR courte.

En vrai

Sur un dépôt de test, crée une branche avec deux commits distincts (deux fichiers différents). Reviens sur `main`, cherry-pick seulement le second. Vérifie que seul le second changement est là. Sens l'outil.

```
git log --oneline -5
git show HEAD
```

Dialogue d'équipe typique

Sam : "Le timeout session casse la prod."

Max : "Le fix est dans ma grosse branche refonte, commit b222."

Sam : "On cherry-pick b222 sur une fix branche, PR express, on livre. Ta refonte continue."

C'est ça, l'outil au service du calendrier. Pas un tour de magie pour éviter le travail : une extraction ciblée.

Dependances entre commits

Si `b222` suppose que `a111` a créé une fonction, cherry-pick de `b222` seul peut casser la compilation. Alors tu cherry-picks `a111` puis `b222`, ou tu recrées un fix minimal sur `main` à la main. Lis le commit avant de cueillir la cerise.

```
git show b222
```

Après le merge du cherry-pick, la feature longue

Quand la refonte de Max arrivera, Git pourra voir des changements déjà présents. Parfois ça se passe bien. Parfois il y a un conflit "déjà appliqué". Respire, résous, continue. Note dans la PR feature : "timeout déjà livré via cherry-pick le ...".

A toi

Ecris dans ton carnet d'équipe : "Cherry-pick = cerise urgente, pas panier entier." Quand la prod brûlera, tu auras la phrase au lieu de la panique.

Chapitre 11 - Bisect : trouver le commit qui a casse

Il y a deux semaines, le formulaire marchait. Aujourd'hui, non. Entre les deux, vingt commits. Personne ne sait lequel a casse. Lire chaque diff a la main est long. `git bisect` fait une recherche par dichotomie : il te fait tester le milieu, puis la moitié restante, jusqu'au premier commit coupable.

L'idée

Tu dis a Git : "la, c'était bon" (un vieux commit). "la, c'est mauvais" (souvent HEAD). Git te pose au milieu. Tu testes. Tu dis bon ou mauvais. Git réduit. En quelques étapes, tu as le commit qui a introduit le bug.

Chez DanielCraft, bisect est le détecteur gentil de l'historique : pas pour blamer une personne, pour trouver un changement.

Le déroulement

```
git bisect start
git bisect bad
git bisect good v1.1.0
```

Ici, tu marques la version taguée `v1.1.0` comme bonne, et l'état actuel comme mauvais. Git checkout un commit du milieu. Tu testes le site (formulaire, page, test auto...).

Si c'est encore casse :

```
git bisect bad
```

Si ça marche :

```
git bisect good
```

Tu repètes. A la fin, Git affiche le premier commit mauvais. Tu lis le diff :

```
git show
```

Puis tu sors du mode bisect :

```
git bisect reset
```

Tu reviens où tu étais. Ensuite tu corriges (souvent une nouvelle branche fix depuis `main`).

Automatiser un peu

Si tu as un test qui échoue de façon fiable, tu peux laisser Git lancer le test à ta place (`git bisect run ...`). Utile, un cran plus avance. Pour commencer, le mode manuel suffit : tu ouvres le navigateur, tu cliques, tu dis bon/mauvais.

Bien choisir good et bad

Il te faut un point vraiment bon. Un tag de release aide. Ou un commit que tu te souviens avoir teste. Si ton "good" n'est pas vraiment bon, bisect te menera nulle part utile.

Bad = l'etat casse. Good = l'etat sain. Ne les inverse pas : tu obtiendrais le message inverse de la realite.

Ce que bisect n'est pas

Ce n'est pas une excuse pour humilier l'auteur du commit. Le commit "coupable" est une info technique. Peut-etre que le bug dependait d'un environnement. Peut-etre que le test manquait. On corrige. On ajoute un test si possible. On avance.

Limites

Si le bug est flaky (parfois oui, parfois non), bisect souffre. Si le bug vient d'une donnee externe, pas du code, bisect ne peut pas inventer la cause. Si plusieurs commits combines causent le probleme, tu trouveras souvent le premier qui fait basculer le test : lis le contexte autour.

Erreur classique

Oublier `git bisect reset` et rester dans un etat etrange. Ou marquer bad/good a l'envers. Ou chercher au hasard pendant une heure alors qu'un tag `v1.1.0` "bon" etait sous la main.

En vrai

Dans un depot de test, introduis volontairement un bug a un commit (par exemple une fonction `addition(a, b)` qui retourne `a - b`), puis ajoute d'autres commits innocents. Lance bisect depuis un bon tag jusqu'a HEAD. Verifie que Git trouve le commit faute. C'est le meilleur tutoriel.

Petite histoire

Mardi, le bouton Envoyer du formulaire ne fait plus rien. Vendredi dernier, demo client OK. Entre les deux, 18 commits (tarifs, analytics, petit refactor JS). Sam lance bisect avec le tag `v0.2.0` comme good. Huit tests manuels plus tard, Git pointe un commit "ajoute tracker clic" qui avait casse le handler. Pas de chasse aux sorcieres : un revert cible ou un fix, plus un test. L'apres-midi est sauvee.

Conseils pour tester pendant bisect

Note ta procedure de test sur un papier (trois clics). Reproduis toujours pareil. Si tu changes de methode au milieu, tu pollues la recherche. Ferme les caches navigateur si besoin. Tu testes le code d'un commit ancien : normal que le site ait l'air "en retard".

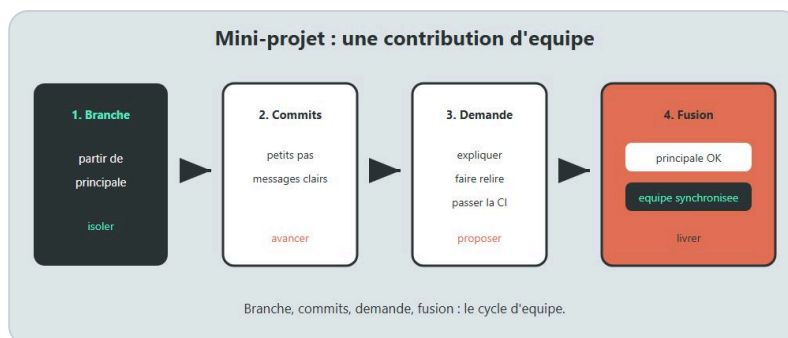
Apres avoir trouve

`git bisect reset`, puis branche fix depuis `main`, correctif, test, PR. Eventuellement ajoute un test automatique pour que la CI attrape ca la prochaine fois. Bisect t'a montre le ou ; la CI evite d'y retourner.

A toi

Ajoute dans tes notes : "Avant de chercher au hasard pendant une heure, je tente bisect si j'ai un point good." Le reflexe compte plus que la syntaxe parfaite.

Chapitre 12 - Mini-projet : simulation d'equipe



Mini-projet : simuler une vraie collab.

Assez de theorie separee. On colle les briques. Tu vas simuler une petite equipe sur un mini site : page d'accueil, page offres, formulaire de contact. Lea, Max, Sam (meme si tu joues les trois roles tout seul).

But : vivre le flux complet une fois, proprement.

Le scenario

Vous livrez `v0.1.0` d'un site vitrine. Puis :

1. Lea ajoute une section tarifs sur la page offres.
2. Max corrige un bug : le formulaire accepte un email invalide.
3. Sam protege `main`, ajoute une CI minimale si possible, et publie `v0.2.0`.

Tu peux tout faire sur un seul compte en changeant de branches et en ouvrant des PR. Idealement, un ami joue le reviewer.

Preparation

Cree un depot GitHub vide (ou local + remote). Clone. Sur `main`, pose un site minimal : `index.html`, `offres.html`, `contact.html`, un peu de CSS. Commit initial clair. Pousse `main`.

```
git switch -c main
# si besoin : premier commit deja sur main
git push -u origin main
```

Active une protection simple sur `main` : PR obligatoire (chapitre 7).

Role Lea : feature tarifs

```
git switch main
git pull
git switch -c feature/section-tarifs
```

Ajoute la section. Commits clairs (contenu, puis style si tu veux). Pousse. Ouvre une PR avec but + comment tester. Fais reviewer (ami ou toi via autre oeil). Merge. Tire `main`.

Role Max : fix email

Pendant ou apres Lea, Max part de `main` a jour :

```
git switch main
git pull
git switch -c fix/validation-email
```

Corrige la validation. Ajoute un test minuscule si tu peux (meme une fonction JS + un runner simple). PR. Review. Merge.

Si la CI existe, regarde le vert. Si elle n'existe pas encore, Sam s'en occupe ensuite, ou tu l'ajoutes dans une PR `chore/ci-legere`.

Role Sam : filets et release

Sam verifie que `main` est protegee. Sam ajoute une CI legere qui lance les tests sur les PR. Sam, apres les merges utiles :

```
git switch main
git pull
git tag -a v0.2.0 -m "Tarifs + validation email"
git push origin v0.2.0
```

Cree une Release GitHub avec trois lignes de notes.

Option bonus : cherry-pick

Si tu as laisse trainer un fix urgent sur une branche longue, rejoue le scenario du chapitre 10 : cerise sur une branche `fix/...`, PR courte, merge.

Option bonus : bisect

Introduis volontairement un bug, ajoute des commits, retrouve-le avec bisect, corrige, tague `v0.2.1`.

Criteres "c'est reussi"

`main` n'a reçu que des merges via PR. Les messages de commit sur les features sont lisibles. Au moins une review a eu lieu (meme simulee). Un tag `v0.2.0` existe. Tu peux expliquer a voix haute le chemin d'une idee jusqu'a la release.

Erreur classique

Tout faire directement sur `main` "pour aller plus vite" : tu rates l'exercice. Ou ouvrir une seule PR geante Lea+Max+Sam. Decoupe. Le mini-projet entraine le decoupage.

En vrai

Chronometre. Une demi-journee suffit souvent. Note ce qui a frotte : protection, conflits, description de PR. Ces frottements sont le cours.

Planning suggere (demi-journee)

0h00-0h30 : socle site + remote + protection main.

0h30-1h30 : role Lea (feature tarifs + PR + merge).

1h30-2h30 : role Max (fix email + test + PR).

2h30-3h15 : role Sam (CI minimale + tag v0.2.0 + notes).

3h15-3h30 : retro ecrite dans le README.

Si tu es seul, enchaîne les casquettes. Si vous êtes trois, prenez les rôles pour de vrai et chronométrez les attentes de review : ça enseigne le "repondre vite".

Definition of done du mini-projet

Une personne extérieure (ami, collègue) peut cloner, lire le README "Comment on travaille", et comprendre comment proposer un changement sans vous poser dix questions. Si oui, vous avez réussi plus que "faire des commandes".

Piege du perfectionnisme

Ne construis pas un design system pendant l'atelier Git. HTML/CSS minimal. L'objet du cours est le flux, pas le pixel perfect. Un bandeau moche versionne proprement bat une refonte visuelle poussée n'importe comment sur `main`.

A toi

A la fin, écris un paragraphe "ce que notre équipe a décidé" : flux, noms de branches, rebase ou merge, qui release. Colle-le dans le README. Le mini-projet devient votre contrat léger.

Chapitre 13 - A retenir

Tu n'as pas besoin de tout memoriser par coeur. Tu as besoin d'une carte mentale stable. Voici la version poche du livre, a relire avant une journee d'equipe.

Le flux

Tire `main` en debut de session. Cree une branche pour une intention. Commit. Pousse la branche. Ouvre une PR. Fais reviewer. Merge. Retire `main`. Recommence. Petit et frequent bat grand et rare.

Les branches

`main` = reference. Feature/fix = travail. Noms clairs. Duree courte. Une intention par branche. Nettoie apres merge. GitHub Flow leger suffit a 2-5 personnes.

Rebase et merge

Merge joint les histoires. Rebase rejoue tes commits sur une base plus recente. Utile sur une feature perso. Dangereux sur l'histoire partagee si tu force-push sans soin. Pas de dogme : une politique d'equipe ecrite.

Historique

Messages qui disent le pourquoi. Petits commits coherents. Squash possible a l'integration. Amend seulement en local / branche perso. Pas de secrets dans Git.

Revue

Bienveillante, utile, rapide. Description claire cote auteur. Regard priorise cote reviewer. Dire aussi ce qui est bien. Pas d'Approve fantome. Pas de tribunal.

Protection et CI

`main` protegee : pas de push direct, PR, review. CI legere : tests sur la PR, feu vert avant merge. Filet technique + filet humain.

Tags et releases

Nommer un commit livre : `vX.Y.Z`. Pousser le tag. Notes courtes. Aligner tag et deploiement.

Cherry-pick et bisect

Cherry-pick : reprendre une cerise (un commit) ailleurs, surtout en urgence. Bisect : trouver le commit qui a casse par recherches successives. Outils de precision, pas de panique.

Contribuer et secrets

Fork + upstream pour contribuer a un projet externe (chapitre 17). Jamais de cles dans le depot (chapitre 18). Les bonnes pratiques d'equipe tiennent en quelques phrases ecrites (chapitre 19).

Phrase DanielCraft

Git en equipe, ce n'est pas "connaître plus de commandes". C'est "se synchroniser sans se blesser". Les commandes servent ce but.

A toi

Sans regarder le livre, écris sur papier les 8 étapes du flux (pull main ... pull main). Si tu bloques, relis le chapitre 2. Si tu y arrives, tu as la colonne vertébrale.

Chapitre 14 - Atelier : flux d'equipe

Cet atelier fait vivre le chapitre 2 avec les mains. Tu joues deux roles : Alex (feature) et Blake (reviewer). Un seul ordinateur suffit.

But

Partir de `main` a jour, creer une branche, pousser, ouvrir une PR, faire merger, revenir sur `main` proprement. Chronometre : 45 a 90 minutes.

Materiel

Un depot GitHub de test. Git configure. Navigateur. Une page `index.html` deja sur `main` (meme minimale).

Etapes

1. 1. Clone le depot (ou ouvre-le) et place-toi sur `main`.

```
git switch main
git pull
```

1. 2. Cree la branche `feature/bandeau-accueil` et ajoute un bandeau texte dans `index.html` ("Bienvenue chez nous").

```
git switch -c feature/bandeau-accueil
```

1. 3. Fais au moins deux commits : un pour le HTML, un pour un peu de CSS si tu veux. Messages clairs.
1. 4. Pousse la branche (pas `main`).

```
git push -u origin feature/bandeau-accueil
```

1. 5. Ouvre une pull request vers `main`. Titre clair. Description avec but et "comment tester" (ouvrir l'accueil, voir le bandeau).
1. 6. En mode Blake, relis le diff. Laisse un commentaire bienveillant (meme mineur). Approuve.
1. 7. Merge la PR (bouton GitHub). Coche la suppression de branche distante si proposee.
1. 8. En local, reviens sur `main`, tire, supprime la branche locale.

```
git switch main
git pull
git branch -d feature/bandeau-accueil
```

1. 9. Verifie que `git status` est propre et que `git log --online -5` montre le merge (ou le squash) attendu.

Variante a deux personnes

Alex fait 1-5. Blake fait 6-7. Alex fait 8-9. Parlez dans un canal : "PR prete", "review ok", "c'est merge".

Criteres de reussite

Aucun push direct sur `main`. La PR a une description. `main` local est a jour apres merge. La branche feature est nettoye.

Si ca bloque

Protection de branche refuse un geste : c'est voulu, passe par la PR. Conflit : tire `main` dans ta feature, resols, pousse. Mauvais remote : verifie `git remote -v`.

Apres l'atelier

Note en trois lignes ce qui t'a freine. Garde la note. Le prochain atelier ira plus vite.

Chapitre 15 - Atelier : revue de code

Ici, le code est un pretexte. L'atelier entraine le regard et le ton. Tu prepareras volontairement une PR "presque bien" avec deux vrais points a discuter.

But

Ecrire une PR aidante, puis faire une revue bienveillante et utile. Duree : 45 a 60 minutes.

Preparation

Sur un depot de test, depuis `main` a jour, cree `feature/formulaire-contact-atelier`.

Ajoute (ou modifie) un formulaire de contact avec :

- un champ email
- un bouton envoyer
- une validation imparfaite (exemple : accepte `a@b` sans domaine complet) - c'est voulu pour la revue
- un commentaire HTML un peu vague `<!-- temp -->` a enlever

Commit en deux temps. Pousse. Ouvre la PR avec une vraie description (but, changements, comment tester). N'avoue pas tous les defaults dans la description : laisse le reviewer travailler.

Etapas auteur

1. Redige le titre : verbe + objet ("Ameliore le formulaire de contact").
2. Remplis but / changements / comment tester.
3. Ajoute une capture si tu peux (meme simple).
4. Demande une review a un ami, ou change de casquette.

Etapas reviewer

1. Lis la description avant le diff.
2. Suis "comment tester" pour de vrai.
3. Laisse au moins deux commentaires utiles : un sur la validation email, un sur le commentaire `temp` (ou autre detail reel).
4. Formule en suggestions, sans insultes, sans "evident".
5. Dis une chose positive (structure, clarte HTML, effort de description...).
6. Choisis : Request changes si le bug email est bloquant, sinon Comment + discussion.

Etapes retour auteur

1. Reponds sans te defendre pour la forme.
2. Corrige la validation. Enleve `<!-- temp -->`.
3. Pousse. Ecris "c'est a jour, tu peux revoir".
4. Fais Approver et merger si c'est bon.

Criteres de reussite

La PR initiale avait une description utilisable. La revue a pointé le fond (validation), pas seulement l'esthétique. Le ton restait respectueux. Un correctif a suivi. `main` a reçu le résultat via merge.

Pieges a eviter

Approuve sans tester. Request changes pour une préférence de virgule. Auteur qui ignore les commentaires. Reviewer qui réécrit toute la PR à la place de l'auteur sans discussion.

Variante solo

Écris la revue dans un fichier `NOTES-REVIEW.md` comme si tu parlais à un collègue. Puis applique tes propres remarques. L'exercice du ton reste valable.

Après l'atelier

Copie deux formulations de commentaires que tu trouves réussies. Elles serviront de modèles la prochaine fois.

Chapitre 16 - Atelier : conflit en equipe



Conflit : lire, choisir, tester, commit.

Les conflits font peur parce qu'ils arrivent souvent le vendredi. Ici, tu vas en créer un exprès, le résoudre calmement, et voir que ce n'est qu'un fichier qui demande un choix.

But

Provoquer un conflit sur le même fichier, le résoudre, finir sur une `main` propre. Durée : 45 à 75 minutes.

Scénario

Deux branches touchent le titre de `index.html` (ou le texte du bandeau). Alex change le titre vers "Atelier Git équipe". Blake change le titre vers "Site vitrine DanielCraft". Les deux partent du même `main`.

Étapes

1. Sur `main` à jour, note le titre actuel.

```
git switch main
git pull
```

1. 2. Branche Alex :

```
git switch -c feature/titre-atelier
```

Modifie le titre. Commit. Pousse. Ouvre une PR. Merge-la dans `main` (avec review rapide ou auto-merge sur dépôt de test).

1. 3. Sans tirer encore, crée la branche Blake depuis l'ancien point... Astuce simple pour solo :

Après le merge d'Alex, crée la branche Blake depuis un commit où le titre n'était pas encore celui d'Alex, OU plus simple pour solo :

```
git switch main
git pull
git switch -c feature/titre-vitrine
```

Modifie le même endroit avec un autre texte. Commit.

1. 4. Ramène `main` dans ta branche Blake :

```
git fetch origin
git merge origin/main
```

Le conflit apparait. Ouvre le fichier. Tu vois les marqueurs :

```
<<<<<< HEAD
titre de Blake
=====
titre d'Alex (venu de main)
>>>>>> ...
```

1. 5. Choisis une version, ou combine ("Site vitrine DanielCraft - atelier Git"). Enleve les marqueurs. Sauve.

1. 6. Marque resolu et termine le merge :

```
git add index.html
git commit
```

(si Git a ouvert un message de merge, valide-le)

1. 7. Pousse. Ouvre la PR. Explique dans la description : "Resolution de conflit sur le titre, version combinee telle." Review. Merge.

1. 8. Tire `main` localement. Verifie le titre final dans le navigateur.

```
git switch main
git pull
```

Variante rebase

Au lieu de `git merge origin/main`, tente `git rebase origin/main`, resols, `git rebase --continue`, puis `git push --force-with-lease` sur la branche perso. Compare le ressenti avec le merge.

Criteres de reussite

Aucun marqueur `<<<<<<` reste dans les fichiers. Le site s'affiche. L'historique montre la resolution. Tu peux expliquer a voix haute ce que tu as choisi et pourquoi.

Si panique

```
git merge --abort
```

ou

```
git rebase --abort
```

Puis recommence. Abandonner proprement est une competence.

Après l'atelier

Ecris la règle d'équipe : "si on touche aux memes zones, on se parle avant" + "on synchronise main souvent". Le meilleur conflit est celui évité tot. Le deuxième meilleur est celui résolu sans drama.

Chapitre 17 - Fork et upstream : contribuer ailleurs

Jusqu'ici, tu étais "dans" le depot de l'équipe : tu avais le droit de pousser des branches. Sur un projet open source (ou le depot d'une autre équipe), tu n'as souvent pas ce droit. Le chemin classique : fork, clone, branche, PR vers le projet original.

Ce chapitre pose le vocabulaire et le geste, sans transformer le livre en encyclopedie GitHub.

Les mots

Fork : une copie du depot sur ton compte GitHub. Tu pousses sur ton fork librement.

Origin : en general, apres clone de ton fork, `origin` pointe vers ton fork.

Upstream : le depot d'origine (celui de l'équipe ou du projet public). Tu le configures pour recuperer les nouveautes, pas pour y pousser directement (tu n'en as souvent pas le droit).

Pull request : tu demandes au projet original d'integrer ta branche (depuis ton fork vers upstream).

ScENARIO concret

Lea veut corriger une typo dans la doc d'un outil qu'elle utilise. Elle ne peut pas pousser sur le depot officiel. Elle fork. Elle clone son fork. Elle cree `fix/typo-readme`. Elle corrige. Elle pousse sur son fork. Elle ouvre une PR vers le depot officiel.

Chez DanielCraft, c'est aussi comme ca qu'on contribue a un outil ami : propre, petit, respectueux des regles du projet (CONTRIBUTING, style, licence).

Les commandes d'installation

Apres avoir fork sur GitHub et clone ton fork :

```
git remote -v
git remote add upstream https://github.com/ORG/PROJET.git
git fetch upstream
```

Remplace l'URL. Verifie :

```
git remote -v
```

Tu dois voir `origin` (ton fork) et `upstream` (l'original).

Rester a jour avec upstream

Avant une nouvelle contribution :

```
git switch main
git fetch upstream
git merge upstream/main
git push origin main
```

(adapte le nom de branche par default si le projet utilise autre chose)

Ainsi ton `main` local et celui de ton fork suivent le projet. Ta prochaine feature part d'une base saine.

Le flux de contribution

```
git switch main
git pull origin main
git switch -c fix/typo-readme
# ... corrections, commits ...
git push -u origin fix/typo-readme
```

Puis sur GitHub : Pull request de `ton-compte:fix/typo-readme` vers `ORG/PROJET:main`.

Lis les commentaires des mainteneurs. Corrige. Sois patient. Une contribution refusee n'est pas une attaque personnelle : parfois le timing, parfois la direction du projet.

Bonnes manieres

PR petite. Sujet unique. Message et description clairs. Respecter le style du projet. Ne pas reformater 200 fichiers "en passant". Ne pas ouvrir dix PR de bruit le premier jour.

Si le projet a un fichier `CONTRIBUTING.md`, lis-le. Ca evite 80% des malentendus.

Et dans une entreprise ?

Parfois tu n'as pas de fork : tu as directement acces au depot, et le chapitre 2 suffit. Le fork sert surtout quand les droits d'ecriture sont limites, ou pour isoler des experiences. Comprendre fork/upstream reste utile des que tu touches l'open source.

Erreur classique

Pousser vers `upstream` alors que tu n'as pas les droits (erreur) ou, pire, croire que `git push` sans remote precise ira "au bon endroit". Verifie. Autre piege : laisser ton fork moisir six mois, puis ouvrir une PR sur une base obsolete : sync d'abord.

En vrai

Fork un petit projet (meme un depot demo), ajoute `upstream`, synchronise `main`, ouvre une PR minuscule sur ton propre fork vers un second depot de test si besoin, pour voir les boutons GitHub. L'important est de voir origin vs upstream une fois dans l'interface.

PR cross-fork : ce que voit le mainteneur

Le mainteneur recoit une PR depuis ton fork. Il voit tes commits, ta discussion, peut demander des changements. Tu pousses sur ta branche de ton fork ; la PR se met a jour. Tu n'as pas besoin d'accès write sur upstream pour iterer.

Quand c'est merge, tu sync ton `main` depuis upstream (fetch + merge) pour rester aligne, et tu peux supprimer ta branche de feature sur ton fork.

Fork pour experimenter

Meme avec acces write, certains forkent pour tenter une idee folle sans polluer les branches de l'equipe. Moins courant en petite boite, courant en open source. L'idee reste : isolation + proposition.

Droits et organisations

Dans une org GitHub, on t'ajoute parfois en write sur le depot : pas besoin de fork. On t'ajoute parfois seulement en lecture + process fork/PR : besoin de fork. Demande quel modele est en place. Ne suppose pas.

A toi

Dessine sur papier trois boites : ton PC, ton fork, le depot original. Fleches : fetch upstream, push origin, PR vers upstream. Si le dessin est clair, le modele est clair.

Chapitre 18 - Hygiene des secrets : jamais de clefs dans Git

Un commit peut vivre longtemps. Il se copie sur les machines, les forks, les backups, les CI logs. Si tu commits une clef API, un mot de passe, un fichier `.env` rempli, tu as publié un secret. Même si tu "effaces" le fichier au commit suivant, l'histoire le contient encore.

Ce chapitre est non négociable. Chez DanielCraft, on préfère un projet un peu moins "pratique à cloner" qu'un projet qui fuit des clés.

Ce qu'on ne commit pas

Fichiers `.env`, `.env.local`, clés privées SSH, fichiers `credentials.json`, dumps de base avec données réelles, exports de mots de passe, tokens de bot, certificats privés.

Les exemples de config, oui. Les vraies valeurs, non.

`.gitignore` est ton ami

Dans le dépôt :

```
.env
.env.*
!.env.example
```

Tu commits `.env.example` avec des fausses valeurs :

```
API_KEY=remplace_moi
DATABASE_URL=postgres://user:pass@localhost:5432/app
```

Chaque développeur copie vers `.env` en local et remplit. Le `.env` reste hors Git.

Comment l'équipe partage les secrets

Pas via un commit "temporaire". Via un gestionnaire (coffre, secrets GitHub, variable d'environnement du serveur, outil d'équipe). Le canal "je te colle la clé sur Discord" est déjà risqué : préfère un outil qui expire, qui audit, qui restreint.

La CI lit les secrets depuis les réglages du dépôt, pas depuis le code.

Si le mal est fait

1. Révoque ou régénère la clé tout de suite chez le fournisseur (cloud, API, DB).
2. Enlève le secret du code actuel.
3. Considère l'historique : selon la gravité, filtre l'historique (outils spécialisés) ou traite le dépôt comme compromis pour cette clé.
4. Préviens l'équipe. Transparence > honte silencieuse.

La priorité absolue : invalider le secret. Un `git rm` seul ne suffit pas.

Revue et secrets

En review, un oeil sur les fichiers suspects : `.pem`, `.env`, `id_rsa`, longs tokens dans le source. Un reviewer qui attrape ca avant le merge sauve des soirees.

Les branches protegees et la CI n'empêchent pas a elles seules un secret dans une PR. L'humain et `.gitignore` comptent.

Exemple Max

Max ajoute l'envoi d'email. Le fournisseur donne une cle. Max met la cle dans `config.js` "pour tester". Il commit. Il pousse. La cle est publique sur le remote. Scenario cauchemar classique. La bonne version : `process.env.MAIL_API_KEY` (ou equivalent) + `.env` ignore + exemple dans `.env.example`.

Erreur classique

"C'est un depot prive, donc OK." Non. Prive aujourd'hui, fork demain, stagiaire apres-demain, fuite un jour. "Je vais l'enlever au prochain commit." L'histoire garde. "C'est juste une cle de dev." Souvent la cle de dev ouvre encore trop.

En vrai

Audit rapide :

```
git ls-files | findstr /i "env pem key credential"
```

(sous Linux/mac : `git ls-files | grep -iE 'env|pem|credential'`)

Verifie `.gitignore`. Tourne les cle qui ont fuit ne serait-ce qu'une fois en local douteux.

Checklist avant le premier push d'une feature API

Est-ce que la cle est dans le code source ? Dans un screenshot de la PR ? Dans un log `console.log` ? Dans un fichier de fixture committe ? Si oui a l'une de ces questions, stop. Sors le secret. Regenere si elle a deja circule.

`.env.example` est un contrat

Il liste les variables necessaires sans valeurs sensibles. Un nouveau developpeur (ou toi sur une nouvelle machine) sait quoi remplir. Si `.env.example` ment (variable manquante), l'onboarding souffre et quelqu'un recommit un vrai `.env` "pour aider". Garde l'exemple juste.

Secrets et captures d'ecran

En review, attention aux screenshots de dashboard cloud avec des cle visibles. Attention aux videos de demo. L'hygiene depasse Git : c'est une posture.

A toi

Ajoute (ou verifie) `.env.example` + regles `.gitignore` sur ton depot de test. Ecris dans le README : "Jamais de secrets dans Git. Les cle vivent hors depot." Lis cette phrase a chaque nouvelle integration d'API.

Chapitre 19 - Bonnes pratiques d'equipe

Les outils ne sauvent pas une equipe qui ne se parle pas. Les pratiques ci-dessous sont le minimum de politesse technique pour 2 a 5 personnes. Adapte. Ecris. Rappel.

Ecrire le contrat leger

Un README section "Comment on travaille" avec : branches, PR obligatoires, qui review, rebase ou merge, comment on release, ou vivent les secrets. Une demi-page. Pas un roman ISO.

Si ce n'est pas ecrit, chacun invente. Puis s'etonne.

Petites PR, souvent

Livre des tranches. Review plus facile. Rollback plus simple. Moral plus haut. La "grosse PR du mois" est un anti-pattern courant.

Synchroniser tot

Tire `main` souvent. Parle si tu touches une zone chaude. Evite les conflits-surprise. Le silence n'est pas de la concentration heroique : parfois c'est de la collision en preparation.

Proteger ce qui compte

`main` protegee. Secrets hors Git. CI legere verte avant merge. Tags pour les versions livrees. Ces quatre gestes evitent une categorie entiere de vendredis tristes.

Review comme un collegue, pas comme un juge

Aide. Questionne. Propose. Remarque le bien. Reponds vite. L'ego hors du diff.

Noms clairs partout

Branches, commits, PR, tags. Le futur lecteur est un humain fatigue a 18h. Ecris pour lui.

Une intention a la fois

Un bug urgent ne voyage pas cache dans une refonte. Deux branches. Deux PR. Le calendrier te remerciera.

Sabbat du force-push

Force-push seulement sur ta branche perso, avec `--force-with-lease`, en conscience. Jamais sur `main`. Jamais "pour voir" sur la branche d'un autre.

Documenter les incidents sans humiliation

Quand ca casse : chronologie, cause, correctif, prevention. Pas de pile au milieu de la piece. Une equipe qui apprend plus vite gagne plus qu'une equipe qui punit.

Chez DanielCraft

On repete souvent : clarte, filets, bienveillance. Git est un moyen. Le produit et les humains sont la fin. Si une pratique Git rend tout le monde miserable sans gain de fiabilite, changez la pratique.

Erreur classique

Copier le process d'une FAANG a trois personnes. Ou n'avoir aucun process. Le juste milieu : quelques regles tenues, pas vingt regles ignorees.

En vrai

Fais une retro de 20 minutes apres deux semaines de mini-projet. Qu'est-ce qui a frotte ? Quelle phrase ajouter au README ? Une seule amelioration suffit par retro.

Rituels hebdomadaires legers

Dix minutes lundi : branches mortes a supprimer ? CI rouge ignoree ? Secrets tourne apres un depart ? Une seule question par semaine suffit a eviter la poussiere.

Onboarding d'un nouveau

Le premier jour, on ne lui demande pas de "lire tout le wiki". On lui donne le README "Comment on travaille", un acces, une petite PR de doc ou de typo pour vivre le flux. La premiere PR est un rituel d'equipe, pas un examen.

Quand casser une regle

Rarement. Explicitement. Annonce dans le canal. Remets la regle apres. Une regle sans exception jamais discutee devient du theatre ; une exception tous les jours devient l'absence de regle.

Mesure simple

Nombre de push directs sur main (doit tendre vers zero). Temps moyen avant premier regard sur une PR. Nombre de secrets revoques apres incident (doit etre "tous", tout de suite). Tu n'as pas besoin d'un dashboard : un ressenti honnete en retro suffit au debut.

A toi

Choisis trois pratiques de ce chapitre et ecris-les comme "non negociables" pour ton equipe. Trois, pas quinze. Les tenir bat les collectionner.

Quiz final

Pas de piège. Relis une fois si besoin. L'idée : vérifier que tu peux avancer en équipe sans paniquer.

Questions

1. 1. Dans un flux feature vers main, que fait-on en général juste avant de créer une branche ?

- A) Supprimer le dépôt distant
- B) Se placer sur `main` et faire `git pull`
- C) Faire `git push --force` sur `main`

1. 2. Une bonne raison de garder des branches feature courtes :

- A) Pour colorier GitHub
- B) Pour limiter les divergences avec `main` et les gros conflits
- C) Parce que Git interdit les branches longues

1. 3. Le rebase, en une idée simple, c'est surtout :

- A) Effacer GitHub
- B) Rejouer tes commits sur une autre base pour un historique plus linéaire
- C) Remplacer `git status`

1. 4. Sur une branche `main` partagée, que faut-il presque toujours éviter ?

- A) Les pull requests
- B) Les tags annotés
- C) Un rebase suivi d'un force push

1. 5. Un message de commit utile raconte surtout :

- A) La liste des fichiers sans intention
- B) Le pourquoi (l'intention) du changement
- C) Le mot de passe de la base

1. 6. Dans une description de PR utile, on trouve souvent :

- A) Uniquement "fix"
- B) But, changements, et comment tester
- C) La clé API en clair

1. 7. Protéger `main` sert surtout à :

- A) Interdire Git localement
- B) Imposer un passage par PR (et souvent review / CI) avant d'intégrer
- C) Ralentir Internet

1. 8. Une CI légère sur les PR, c'est surtout pour :

- A) Remplacer toute review humaine

- B) Lancer automatiquement des verifs (tests, lint, build) et signaler rouge/vert

- C) Generer des logos

1. 9. Un tag `v1.2.0` sert surtout a :

- A) Remplacer les branches
- B) Nommer de facon stable un commit (souvent une version livree)
- C) Effacer l'historique

1. 10. `git cherry-pick` sert surtout a :

- A) Copier un commit precis sur une autre branche
- B) Supprimer GitHub
- C) Remplacer `.gitignore`

1. 11. `git bisect` sert surtout a :

- A) Punir un collegue en public
- B) Trouver par recherches successives le commit qui a introduit un bug
- C) Merger automatiquement

1. 12. La bonne hygiene pour une cle API :

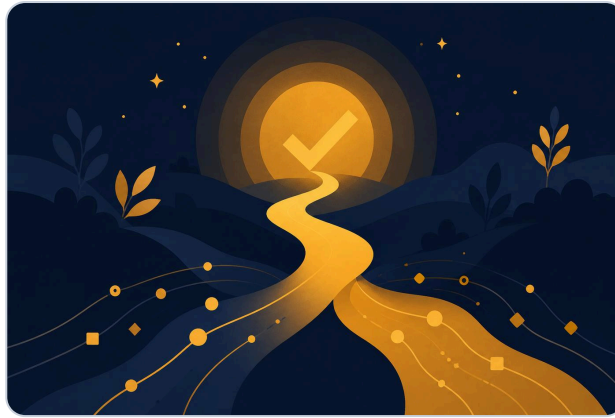
- A) La commit dans `config.js` "temporairement"
- B) La mettre hors Git (ex: `.env` ignore) et fournir un `.env.example`
- C) L'ecrire dans le titre de la PR

Corriges

1-B, 2-B, 3-B, 4-C, 5-B, 6-B, 7-B, 8-B, 9-B, 10-A, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es pret pour de vraies collabos. Sinon, relis les chapitres lies (flux, rebase, revue, protection, CI, tags, cherry-pick, bisect, secrets), sans dramatiser.

Chapitre 21 - Bravo



Bravo. Tu sais travailler Git en equipe.

Tu as fini Git en equipe. Pas "tu connais toutes les options de toutes les commandes". Mieux : tu sais collaborer sans te marcher dessus.

Tu as une carte. Flux. Branches simples. Rebase et merge sans dogme. Historique lisible. Revue humaine. `main` protegee. CI legere. Tags et releases. Cherry-pick pour une cerise urgente. Bisect pour trouver le commit casse. Un mini-projet. Des ateliers. Fork et upstream. Secrets hors du depot. Des pratiques d'equipe tenables.

Chez DanielCraft, on mesure le progres a une chose simple : est-ce que ton prochain projet a plusieurs sera plus calme que le precedent ? Si oui, le livre a servi.

Ce que tu peux faire maintenant

Tenir un README de collaboration. Ouvrir des PR petites et claires. Reviewer sans blesser. Refuser le push direct sur `main`. Taguer une version. Retrouver un bug dans l'histoire. Contribuer via fork. Garder les cles dehors.

La suite

La suite, c'est la vraie vie : un depot, des humains, des vendredis. Applique une pratique a la fois. Garde le contrat leger a jour. Quand ca frotte, ajustez sans ego.

Si tu veux aller plus loin plus tard : hooks avances, strategies de monorepo, automatisation de changelog, deploiements lies aux tags. Ce livre t'a donne le socle d'equipe. Le reste s'accroche dessus.

Une derniere phrase

Pousse ta branche. Demande un regard. Merci ton reviewer. Tire `main`. Recommence.

Bravo. Tu sais travailler a plusieurs avec Git.

Tu as les briques. Maintenant, construis.